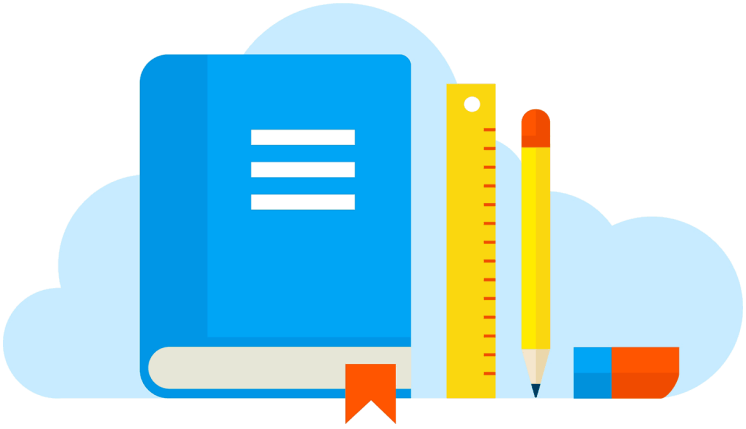


📘 NOTE 2 — JAVASCRIPT FUNDAMENTALS

Date: 09/04/2026

JAVASCRIPT FUNDAMENTALS

Web Development Notes

Subject: Web Development
Topic: JavaScript Fundamentals
Level: BCA Semester 5

---

1. Introduction to JavaScript

JavaScript is a high-level programming language used to make web pages interactive and dynamic.

HTML creates the structure of a webpage, CSS provides the design, and JavaScript adds **logic and interactivity**.

JavaScript can be used to:

- Validate forms
- Handle button clicks
- Change webpage content
- Display messages
- Perform calculations
- Create animations
- Fetch data from servers
- Build interactive web applications

For example, JavaScript can check whether a student has entered a valid email address before submitting a registration form.

---

2. Features of JavaScript

Important features of JavaScript include:

1. **Lightweight** – It is relatively simple and fast.
2. **Interpreted** – JavaScript code can be executed by the browser.
3. **Object-oriented** – It supports objects and object-based programming.
4. **Dynamic** – Variables can hold different types of values.
5. **Event-driven** – It can respond to user actions such as clicks and keyboard input.
6. **Cross-platform** – It works on different operating systems and browsers.
7. **Client-side execution** – JavaScript can execute directly in the user's browser.
8. **Supports asynchronous programming** – It can perform tasks such as API requests without blocking the entire webpage.

---

3. Adding JavaScript to HTML

There are three common ways to add JavaScript to a webpage.

Inline JavaScript

JavaScript is written directly inside an HTML element.

```
<button onclick="alert('Welcome to StudySphere!')">
    Click Me
</button>
```

Internal JavaScript

JavaScript is written inside the `<script>` tag.

```
<script>
    let message = "Welcome to StudySphere";
    console.log(message);
</script>
```

External JavaScript

JavaScript is stored in a separate `.js` file.

```
<script src="script.js"></script>
```

External JavaScript is useful for larger projects because it keeps HTML and JavaScript code separate.

---

4. Variables

Variables are used to store data in a program.

JavaScript provides three commonly used keywords for declaring variables:

- `var`
- `let`
- `const`

Example

- let studentName = "Aarav";
- let semester = 5;

const course = "BCA";

Here:

- `studentName` stores the student's name.
- `semester` stores the semester number.
- `course` stores a constant value.

Difference Between `let` and `const`

`let` is used when the value may change.

- let marks = 70;

marks = 85;

`const` is used when the value should not be reassigned.

const college = "SDJ International College";

---

5. Data Types

A data type defines the kind of value stored in a variable.

Common JavaScript data types are:

String

Used to store text.

let name = "Aarav";

Number

Used to store numeric values.

let marks = 85;

Boolean

Stores either `true` or `false`.

let isSubmitted = true;

Undefined

A variable that has been declared but has not been assigned a value.

let result;

Null

Represents an intentional empty value.

let grade = null;

Object

Used to store related data in key-value form.

- let student = {
- name: "Aarav",
- semester: 5

};

6. Operators

Operators are symbols used to perform operations on values.

Arithmetic Operators

- \+ Addition
- \- Subtraction
- * Multiplication
- / Division

% Modulus

Example:

- let a = 20;
- let b = 5;
-
- console.log(a + b);

console.log(a * b);

Comparison Operators

- == Equal
- === Strictly equal
- != Not equal
- > Greater than
- < Less than
- >= Greater than or equal

<= Less than or equal

Example:

- let marks = 75;
-

console.log(marks >= 40);

Output:

true

Logical Operators

- && AND
- || OR

! NOT

Example:

- let marks = 75;
- let attendance = 80;
-
- if (marks >= 40 && attendance >= 75) {
- console.log("Student is eligible");

}

---

7. Conditional Statements

Conditional statements are used to execute different code based on a condition.

if Statement

- let marks = 75;
-
- if (marks >= 40) {
- console.log("Pass");

}

if-else Statement

- let marks = 35;

```
○
○ if (marks >= 40) {
○     console.log("Pass");
○ } else {
○     console.log("Fail");
}
```

else-if Statement

```
○ let marks = 85;
○
○ if (marks >= 80) {
○     console.log("Grade A");
○ } else if (marks >= 60) {
○     console.log("Grade B");
○ } else {
○     console.log("Grade C");
}
```

---

8. Loops

Loops are used to execute a block of code repeatedly.

for Loop

```
○ for (let i = 1; i <= 5; i++) {
○     console.log(i);
}
```

Output:

○ 1
○ 2
○ 3

- 4

5

while Loop

```
let i = 1;

while (i <= 5) {
    console.log(i);
    i++;
}
```

Loops are useful when working with lists of students, notes, assignments, or other repeated data.

---

9. Functions

A function is a reusable block of code designed to perform a specific task.

Basic Function

```
function greet() {
    console.log("Welcome to StudySphere");
}

greet();
```

Function with Parameters

```
function add(a, b) {
    return a + b;
}

let result = add(10, 20);
```

```
console.log(result);
```

Output:

```
30
```

Functions help make programs **organized, reusable, and easier to maintain**.

---

10. Arrays

An array is used to store multiple values in a single variable.

```
let subjects = [
    "Web Development",
    "DBMS",
    "Computer Networks",
    "Operating Systems"
];
```

Array elements are accessed using their index.

```
console.log(subjects[0]);
```

Output:

```
Web Development
```

JavaScript provides useful array methods such as:

- `push()` – adds an element
- `pop()` – removes the last element
- `shift()` – removes the first element
- `unshift()` – adds an element at the beginning
- `map()` – creates a new array
- `filter()` – filters elements

- `forEach()` – executes a function for each element

---

11. Objects

Objects store related information using **key-value pairs**.

```
let student = {
    name: "Aarav",
    course: "BCA",
    semester: 5,
    email: "aarav@example.com"
};
```

Accessing object data:

```
console.log(student.name);
console.log(student.course);
```

Objects are very important in modern web development because data received from APIs is commonly represented in object or JSON format.

---

12. DOM

DOM stands for Document Object Model.

The DOM represents an HTML document as a structure of objects. JavaScript can use the DOM to access and modify webpage elements.

For example:

```
<h2 id="title">Welcome</h2>
```

JavaScript can change the text:

- document.getElementById("title").textContent =

  "Welcome to StudySphere";

The webpage will then display:

Welcome to StudySphere

---

13. Events

An event is an action that occurs on a webpage.

Examples:

- Mouse click
- Keyboard input
- Form submission
- Page loading
- Mouse movement

Example:

- <button id="btn">Click Me</button>
-
- <script>
- document.getElementById("btn").addEventListener("click", function() {
- alert("Button clicked!");
- });

</script>

Events are important for creating interactive web applications.

---

14. Form Validation

JavaScript can be used to validate user input before sending it to the server.

For example, we can check:

- Required fields
- Email format
- Password length
- Password confirmation
- Valid numbers
- Empty fields

Example:

```
let email = document.getElementById("email").value;

if (email === "") {
    alert("Email is required");

}
```

In a project such as **StudySphere**, JavaScript can help validate registration, login, and assignment submission forms.

---

15. JavaScript in StudySphere

JavaScript can be used in StudySphere for:

- Registration form validation
- Login form validation
- Searching notes
- Filtering notes by subject
- Handling buttons and menus
- Dynamic dashboard interactions
- Assignment submission interactions
- Displaying messages

- Communicating with backend APIs

In the MERN version of StudySphere, JavaScript is especially important because **React and Node.js are both JavaScript-based technologies**.

---

16. Conclusion

JavaScript is an important technology in modern web development. It adds logic and interactivity to websites and allows developers to create dynamic applications.

By learning variables, data types, operators, conditions, loops, functions, arrays, objects, DOM, and events, a developer can build the foundation required for advanced technologies such as **React.js and Node.js**.